

Epic / EPIC SecureMsg cryptographic design document ▾

You are helping write a Cryptographic Design Document for a university project called EPIC (SecureMsg), a group end-to-end encrypted messaging application. This document is assessed by a cryptography examiner and is worth 5% of the project mark. The rubric says an Excellent document "reads like a small security audit."

The document must be 4–6 pages, in Markdown, and cover the following sections exactly:

1. Threat Model — one paragraph per attacker class:
 - (a) passive network attacker, (b) active network attacker, (c) honest-but-curious server, (d) fully compromised server. State explicitly what breaks under (d).
2. Key Material Overview — a table of every key in the system: IK (X25519), IK signing key (Ed25519), SPK (X25519), OPKs (X25519), EK (X25519), ratchet keypairs, message keys (MK), chain keys (CK), root key (RK), session wrap key. For each: how generated, where it lives, when it's erased.
3. Registration & Key Publication Flow — diagram (ASCII) + prose. Cover: identity generation, Argon2id wrap at rest, SPK signing with Ed25519, OPK batch upload, server-side SPK signature verification.
4. Send & Receive Flow — ASCII diagram + prose. Cover: X3DH handshake (4 DH operations, HKDF-SHA256 with info=SecureMsg_X3DH_v1, salt=32 zero bytes), Double Ratchet (symmetric chain step with info=SecureMsg_MsgKey_v1, DH ratchet step with info=SecureMsg_Ratchet_v1), AAD construction (IK_A || IK_B || ratchet_pub || PN || N), nonce strategy (random 96-bit per message, reuse impossible because each MK is derived once and used once), PKCS#7 padding to 256-byte boundary.
5. Primitive Justification Table — one row per primitive with columns: Primitive, Algorithm + Params, Security Property, Why These Params, RFC/Citation. Must cover:
 - AES-256-GCM (NIST SP 800-38D, RFC 5116)
 - X25519 / DHKEM (RFC 7748)
 - Ed25519 (RFC 8032)
 - HKDF-SHA256 (RFC 5869)
 - Argon2id server-side: m=65536 KiB, t=3, p=4 (RFC 9106, OWASP)
 - Argon2id client-side key wrap: m=65536 KiB, t=3, p=1 — explain why p differs (libsodium crypto_pwhash API fixes p=1; deviation acknowledged, negligible for threat model)
 - X3DH protocol (Signal X3DH spec §3–§4)
 - Double Ratchet (Signal DR spec §3, §4.2 skipped keys)
6. Trust Model — TOFU (Trust On First Use). Explain: first-contact key pinning via server-stored public keys, Ed25519 SPK signature verification

on every send, known limitation (first-contact MITM window before pinning), why PKI was not used (complexity, no CA infrastructure, TOFU is standard for this threat model — cite Signal).

7. Known Limitations — honest list including: TOFU first-contact MITM window, metadata (who talks to whom) visible to server, skipped message keys cached in memory (bounded at MAX_SKIPPED=100), identity wrap key resident in memory for session lifetime, no cross-device account portability by design (private keys are device-local, non-extractable — this is a trust model strength not a limitation), OPK depletion fallback drops to 3-DH X3DH.

Implementation notes to weave in where relevant:

- Python crypto daemon handles all crypto; C++ Qt client sends opaque ciphertext via IPC — no private key material in the C++ layer
- Browser frontend uses Web Crypto API, same protocol, interoperable
- Private keys stored as non-extractable CryptoKey in IndexedDB (browser) and Argon2id+AES-256-GCM encrypted files on disk (daemon)
- X25519 is used for all DH operations; Ed25519 is signing-only (SPK signature) — these must not be conflated
- SHA-256 used only for fingerprints/diagnostics, never in security-relevant roles

Output the full document in Markdown, ready to save as crypto-design.md. Use RFC section number citations inline (e.g. RFC 5869 §2.2). Write at parameter level throughout — "it's standard" is not a justification.

Show less

Reconciling rubric requirements with user's Signal protocol implementation >

I'll quickly check the cryptography guide to make sure the document aligns with the examiner's rubric, then write the full document.

Reconciled protocol specifications with rubric requirements, structured comprehensive audit doc... >

The cryptography guide confirms exactly what the rubric rewards — parameter-level justification, all four attacker classes, honest limitations, and info-string domain separation. Your spec already aligns with it. I'll write the full document now and add two low-cost "Excellent-tier" touches the guide flags (key-commitment awareness, since forwarding is a feature, and a post-quantum note) inside the Known Limitations section, where they fit honestly.

Full cryptographic design document for EPIC/SecureMsg, written to outputs as crypto-design.md >

continue

Presented file >

Done — `crypto-design.md` covers all seven sections at parameter level, with RFC section citations inline and ASCII diagrams for the registration and send/receive flows.

A few things worth knowing before you submit:

Where it goes beyond the spec (to hit the Excellent tier the guide describes):

- Added a key-commitment / AES-GCM-SIV awareness bullet in §7, since forwarding is a feature and the guide flags this as a mark-winner.
- Added a post-quantum (harvest-now-decrypt-later) limitation with the X25519+ML-KEM future-work path — also explicitly rewarded by the rubric.
- Listed your three actual info strings and explained pairwise domain separation, which the guide calls the difference between "Good" and "Excellent."

How it sets up Eoin's six interview probes (each is now defensible from the doc):

- SPK signature verification → §3 (server-side) + §4 (re-verified by every sender before any DH)
- DoS on skipped keys → §7 MAX_SKIPPED=100 bullet ties the cap to the skip-count attack
- AEAD auth-failure handling → §1(b), §4 (tampered tag/AAD rejected, padding inside the AEAD)
- Nonce handling → §4 articulates the structural argument (one key per message → reuse impossible regardless of collision probability)
- Ratchet state at rest → §2 table + §3 wrap, with the browser non-extractable vs daemon Argon2id distinction
- IK dual-use → handled by framing IK (X25519) and IK_sig (Ed25519) as **separate** keypairs with no XEdDSA conversion, and stating explicitly they're never conflated

Two things to verify against your code before submitting, since I wrote to the spec rather than reading the daemon:

1. The DH-ratchet KDF input — I wrote `HKDF(RK || DH_out, info="SecureMsg_Ratchet_v1")`. Confirm your `double_ratchet.py` passes `DH_out`

as the salt vs. concatenated IKM; Signal's spec uses RK as salt and DH_out as IKM, so if yours follows that, flip the wording in §4.

- Whether the at-rest wrap actually calls libsodium `crypto_pwhash` (p=1) in the Python daemon, or argon2-cffi. The p=1-vs-p=4 justification in §5 hinges on it being libsodium — if it's argon2-cffi with p=1 set manually, the rationale should say "we fix p=1 to match the libsodium client" rather than "the API fixes it."

If you paste me the relevant lines from `double_ratchet.py` and the key-wrap function, I'll reconcile those two details exactly.

Crypto design
Document · MD

Download

excerpt_from_previous_claude_message.txt

1 line

TXT

```
def dh_ratchet_hkdf(rk: bytes, dh_out: bytes) -> Tuple[bytes, bytes]:
    """(RK_next, CK_x) = HKDF(DH, salt=RK, info=RATCHET_INFO)
    -> 64 bytes."""
    out = _hkdf(dh_out, 64, rk, RATCHET_INFO)
    return out[:32], out[32:]
```

Identified documentation error in HKDF parameter ordering and flagged remaining verification item >

The code answers question 1 cleanly — and confirms my doc had it backwards. Your implementation uses `salt=RK, IKM=DH_out` (the correct Signal-spec convention), whereas I wrote it as a concatenation `RK || DH_out`. Let me fix that.

Hunted down section four's prose and diagram components >

Now I'll fix the prose (line 134), which had the inputs concatenated rather than salt/IKM-separated:

Edited 2 files >

Now the diagram (line 122–123), which was ambiguous:

Edited a file, read a file >

Fixed. §4 now matches your `dh_ratchet_hkdf` exactly: `HKDF(ikm=DH_out, salt=RK, info="SecureMsg_Ratchet_v1")` → 64 bytes split into `RK_next` (first 32) and the new chain key (last 32), in both the diagram and the prose. I also noted that using RK as the salt (rather than concatenating it) is the deliberate Signal-spec convention — which is a good thing to be able to say in the interview, since it shows you know *why* it's structured that way and not just that it works.

One detail your code surfaces that's worth a quick sanity check on your side: the docstring calls the outputs `(RK_next, CK_x)` and returns `out[:32], out[32:]`. The doc assumes first-32 = root key, last-32 = chain key. That ordering matches your return statement, so you're consistent — just confirm the *caller* unpacks it the same way (`rk, ck = dh_ratchet_hkdf(...)`) and doesn't swap them.

Still open from before: **question 2** — whether the at-rest key wrap actually goes through libsodium `crypto_pwhash` (p hard-fixed at 1) or through argon2-cffi with p=1 set by hand. §5 currently says "libsodium's `crypto_pwhash` API fixes the lanes parameter." If it's argon2-cffi, that one clause needs to change to "we fix p=1 to match the libsodium-based client wrap" — otherwise Eoin could catch the claim that the API forces it when it doesn't. Paste the wrap/derive function and I'll align it.

Crypto design
Document · MD

Download

excerpt_from_previous_claude_message.txt

1 line

TXT

where can i check this

Charted PowerShell commands to locate cryptographic library usage >

The fastest place to check is your local daemon source. Two things tell you which library it is: what's installed, and what the wrap function imports/calls. From

PowerShell:

```
powershell

cd C:\Users\m4tth\EPIC\crypto-daemon

# 1. Which library is even installed?
Select-String -Path requirements*.txt,pyproject.toml -Pattern "nacl|argon|libsod

# 2. Where Argon2 is actually called (import + call site)
Get-ChildItem -Recurse -Filter *.py | Select-String -Pattern "pwhash|argon|hash_
```

The second command will land you on the wrap/derive function and its import line. That's the whole answer — read the import and the call:

- **libsodium (PyNaCl):** imports look like `from nacl import pwhash` / `nacl.pwhash.argon2id.kdf(...)` / `crypto_pwhash`. PyNaCl's high-level KDF takes only `opslimit` and `memlimit` — there is no parallelism argument, because libsodium hardcodes lanes = 1. If this is what you see, the §5 wording ("the API fixes the lanes parameter") is **correct as written**.
- **argon2-cffi:** imports look like `from argon2 import ...`, `from argon2.low_level import hash_secret_raw, Type`, or `PasswordHasher(...)`. These expose a `parallelism=` argument that you set. If this is what you see, look at that argument's value — and change the §5 clause to "we set p=1 to match the libsodium-based client wrap" rather than claiming the API forces it.

If the daemon is the one doing the wrap (it should be — no private keys in the C++ layer per your design), this is where it'll be. If command 2 returns nothing under `crypto-daemon`, the wrap may live in the C++ client instead, in which case search there for `crypto_pwhash` (libsodium in C++ always fixes p=1, so the original wording would hold):

```
powershell

cd C:\Users\m4tth\EPIC
Get-ChildItem -Recurse -Include *.cpp,*.h | Select-String -Pattern "crypto_pwhas
```

Paste whichever function comes back and I'll set the §5 clause to match exactly.

